

Tarea 3: Simulación Estocástica

Pablo León Evia

2026-09-09

Se presentará los procedimientos y el código para resolver la tarea 3.

1. Considera una variable aleatoria X con distribución Gamma de parámetros $(\alpha, 1)$ suponiendo $\alpha \geq 1$ y realiza lo siguiente:

(a) Muestra que $\frac{X}{\beta}$ se distribuye gamma de parámetros (α, β) . ¿Qué significa esto para efectos de la simulación de variables gamma? Para realizar la transformación, me apoyo de la función de distribución de la variable aleatoria Y .

$$F_Y = (Y \leq y) = P\left(\frac{X}{\beta} \leq y\right) = P(X \leq \beta y) = F_X(\beta y)$$

Derivando con respecto a Y tenemos:

$$\implies f_Y(y) = f_X(\beta y)\beta = \frac{\beta y^{\alpha-1} e^{-\beta y}}{\Gamma(\alpha)}$$

$$\implies Y \sim \text{Gamma}(\alpha, \beta)$$

(b) Para el caso $\alpha < 1$ muestra que si X es Gamma($1+\alpha, 1$) independiente de $U \sim U(0, 1)$ entonces $XU^{1-\alpha}$ se distribuye Gamma($\alpha, 1$). La variable aleatoria conjunta de X, U es igual a la $f_X(x)$ por independencia. Hacemos los siguientes cambios de variables:

$$Y = XU^{\frac{1}{\alpha}} \implies U = \left(\frac{Y}{V}\right)^{\alpha}$$

Teniendo en cuenta que:

$$V = X$$

Construimos la matriz Jacobiana como:

$$J = \begin{bmatrix} 0 & 1 \\ \frac{\alpha y^{\alpha-1}}{v^{\alpha}} & \frac{-\alpha y^{\alpha}}{v^{\alpha+1}} \end{bmatrix}$$

$$\det(J) = 0 \cdot \left(\frac{-\alpha y^{\alpha}}{v^{\alpha+1}}\right) - 1 \cdot (\alpha y^{\alpha-1} v^{-\alpha})$$

Sabemos que la $V \geq 0$ y la $0 \leq U \leq 1$ implica que $Y, V \geq 0$.

$$\implies |J| = \alpha y^{\alpha-1} v^{-\alpha}$$

Tenemos que la función de densidad conjunta queda definido por:

$$f_{Y,V}(y, v) = f_{X,U}(v, \left(\frac{y}{v}\right)^{\alpha}) \cdot |J|$$

Reemplazamos:

$$f_{Y,V}(y, v) = \frac{v^\alpha e^{-v}}{\Gamma(1+\alpha)} \cdot \frac{\alpha y^{\alpha-1}}{v^\alpha} = \frac{y^{\alpha-1} e^{-v}}{\Gamma(\alpha)}; 0 < y < v < \infty$$

Como queremos la función marginal de Y tenemos:

$$f_Y(y) = \int_y^\infty \frac{1}{\Gamma(\alpha)} y^{\alpha-1} e^{-v} dv = \frac{y^{\alpha-1}}{\Gamma(\alpha)} \cdot [-e^{-v}]_y^\infty$$

Evaluamos el límite y tenemos:

$$f_Y(y) = \frac{y^{\alpha-1}}{\Gamma(\alpha)} \left(-\lim_{v \rightarrow \infty} e^{-v} - (-e^{-y}) \right)$$

$$f_Y(y) = \frac{y^{\alpha-1}}{\Gamma(\alpha)} (0 + e^{-y}) = \frac{1}{\Gamma(\alpha)} y^{\alpha-1} e^{-y}$$

Se ve claramente que la variable aleatoria Y se distribuye $\text{Gamma}(\alpha, 1)$.

(c) Implementa un algoritmo de aceptación y rechazo para simular de una distribución gamma de parámetros $(\alpha, 1)$ (con $\alpha \geq 1$) utilizando como distribución instrumental una gamma de parámetros $(\lfloor \alpha \rfloor, \frac{\lfloor \alpha \rfloor}{\alpha})$. Tenemos dos variables aleatorias: $X \sim \text{Gamma}(\alpha, 1)$ y $Y \sim \text{Gamma}(\lfloor \alpha \rfloor, \frac{\lfloor \alpha \rfloor}{\alpha})$ donde las densidades son:

$$f : \frac{x^{\alpha-1} e^{-x}}{\Gamma(\alpha)}$$

$$g : \frac{\left(\frac{\lfloor \alpha \rfloor}{\alpha}\right)^{\lfloor \alpha \rfloor}}{\Gamma(\lfloor \alpha \rfloor)} x^{\lfloor \alpha \rfloor - 1} e^{-\frac{\lfloor \alpha \rfloor}{\alpha} x}$$

Para no excedernos se pondrá el valor encontrado de m y la regla de decisión:

$$c = \frac{\Gamma(\lfloor \alpha \rfloor)}{\Gamma(\alpha)} \left(\frac{\alpha}{\lfloor \alpha \rfloor} \right)^{\lfloor \alpha \rfloor} \alpha^{\alpha - \lfloor \alpha \rfloor} e^{-(\alpha - \lfloor \alpha \rfloor)}$$

Se acepta el valor si se cumple que:

$$U \leq \frac{f(Y)}{c \cdot g(Y)} = \left(\frac{Y}{\alpha} \right)^{\alpha - \lfloor \alpha \rfloor} \exp \left\{ - \left(1 - \frac{\lfloor \alpha \rfloor}{\alpha} \right) (Y - \alpha) \right\}$$

donde: $U \sim (0, 1)$ Sabemos que la función piso justamente hará que mi función Gamma tenga un parámetro entero, lo que nos permitirá obtener la función Gamma a partir de la suma de exponenciales, para agilizar el código se harán con la función `rgamma` pero la idea para obtenerlo ya se mencionó.

```
# Las entradas de la función es el alpha y el número de muestras requeridas.
set.seed(325325)
Func_gamma <- function(alpha, n) {
  k <- floor(alpha)
  alfota <- k/alpha

  # Constante c
  c <- (gamma(k) / gamma(alpha)) * (alpha / k)^k * alpha^(alpha - k) * exp(-(alpha - k))
  # Creamos una cadena vacía y un contador de intentos para llevar un
  # registro de la eficiencia.
  aceptados <- c()
  total_intentos <- 0
```

```

while (length(aceptados) < n) {
  # 1. Generamos de golpe los datos que nos faltan
  faltantes <- n - length(aceptados)

  # Simulamos gammas y uniformes(0,1)
  Y <- rgamma(faltantes, shape = k, rate = alfota)
  U <- runif(faltantes)

  # Vectorizamos el criterio
  radio <- (Y / alpha)^(alpha - k) * exp(-(1 - alfota) * (Y - alpha))

  # Filtramos por la condición de aceptación
  nuevos_aceptados <- Y[U <= radio]
  aceptados <- c(aceptados, nuevos_aceptados)
  total_intentos <- total_intentos + faltantes
}

# Debido a que el filtro de cantidad faltante se hace en cada ciclo
# While, nos quedamos con los primeros n datos
muestra <- aceptados[1:n]

# Comparamos las eficiencias
efi_teo <- 1/c
efi_obs <- n/total_intentos

# Regresamos varios valores
return(list(
  muestra = muestra,
  efi_teo = efi_teo,
  efi_obs = efi_obs,
  alpha = alpha)
)
}

# Prueba
prueba <- Func_gamma(alpha = 9.3464, n=10000)
resultado_ks <- ks.test(x = prueba$muestra, y = "pgamma", shape = prueba$alpha, rate = 1)

cat("De la función Gamma creada a partir del método de aceptación\n",
    "y rechazo se obtuvo una muestra de 10,000 datos.\n",
    "Eficiencia teórica vs observada: ", prueba$efi_teo, " vs ", prueba$efi_obs, "\n",
    "Prueba KS: p-value = ", resultado_ks$p.value, "\n")

```

```

## De la función Gamma creada a partir del método de aceptación
## y rechazo se obtuvo una muestra de 10,000 datos.
## Eficiencia teórica vs observada: 0.9809575 vs 0.9797198
## Prueba KS: p-value = 0.8176233

```

(d) Muestra que si X se distribuye gamma $(\alpha, 1)$ y Y se distribuye gamma $(\beta, 1)$, entonces $\frac{X}{X+Y}$ se distribuye beta de parámetros (α, β) Suponiendo que X y Y son independientes, tenemos la densidad conjunta como:

$$f_{X,Y}(x,y) = \frac{1}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1} y^{\beta-1} e^{-(x+y)}, \quad x > 0, y > 0$$

Proponemos el cambio de variable:

$$V = \frac{X}{X+Y}, \quad Z = X+Y$$

Despejando X e Y :

$$X = VZ, \quad Y = Z(1-V)$$

El soporte para las nuevas variables es $0 < v < 1$ y $z > 0$. Calculamos el Jacobiano de la transformación:

$$J = \begin{bmatrix} z & v \\ -z & 1-v \end{bmatrix}$$

Calculamos el determinante de la matriz Jacobiana

$$\det(J) = z(1-v) - (-zv) = z - zv + zv = z \implies |\det(J)| = z$$

Aplicando el cambio de variable a la densidad conjunta:

$$f_{V,Z}(v, z) = f_{X,Y}(vz, z(1-v)) \cdot |\det(J)|$$

Sustituyendo:

$$f_{V,Z}(v, z) = \left(\frac{1}{\Gamma(\alpha)} (vz)^{\alpha-1} e^{-vz} \right) \cdot \left(\frac{1}{\Gamma(\beta)} (z(1-v))^{\beta-1} e^{-z(1-v)} \right) \cdot z$$

$$f_{V,Z}(v, z) = \frac{1}{\Gamma(\alpha)\Gamma(\beta)} v^{\alpha-1} (1-v)^{\beta-1} z^{\alpha+\beta-1} e^{-z}$$

Calculamos la densidad marginal de V integrando respecto a z :

$$f_V(v) = \frac{1}{\Gamma(\alpha)\Gamma(\beta)} v^{\alpha-1} (1-v)^{\beta-1} \int_0^\infty z^{\alpha+\beta-1} e^{-z} dz$$

Sustituyendo el valor de la integral:

$$f_V(v) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} v^{\alpha-1} (1-v)^{\beta-1}, \quad 0 < v < 1$$

$$\implies V = \frac{X}{X+Y} \sim \text{Beta}(\alpha, \beta)$$

Sabemos que la función Beta se puede expresar como el cociente de las funciones Gamma. (e) Utiliza los resultados anteriores y tu código para simular una muestra de tamaño $n = 100, 1000, 10000$ de una beta de parámetros $(2.2, 4.7)$. Para cada tamaño de muestra analiza la eficacia del algoritmo en términos del número total de muestras propuestas y el número total de muestras rechazadas. ¿Qué tan eficiente es el algoritmo? Se usará la función creada con anterioridad para el cálculo de una beta.

```
# 1. Función Beta ajustada con conteo de intentos correcto
func_beta <- function(alpha, beta, n) {
  X <- Func_gamma(alpha, n)
  Y <- Func_gamma(beta, n)

  V <- X$muestra / (X$muestra + Y$muestra)

  # Estimación de la eficiencia
  propuestas_X <- n / X$efi_obs
  propuestas_Y <- n / Y$efi_obs

  propuestas_totales <- propuestas_X + propuestas_Y
  muestras_rechazadas <- propuestas_totales - (2 * n)
  eficiencia_observada <- (2 * n) / propuestas_totales
```

```

return(list(
  V = V,
  alpha = alpha,
  beta = beta,
  propuestas = propuestas_totales,
  rechazos = muestras_rechazadas,
  efi_global = eficiencia_observada
))
}

# 2. Prueba individual para n = 100
prueba <- func_beta(alpha = 2.2, beta = 4.7, n = 100)
resultado_ks <- ks.test(x = prueba$V, y = "pbeta",
  shape1 = prueba$alpha,
  shape2 = prueba$beta)

cat("De la función Beta creada a partir del método de aceptación\n",
  "y rechazo se obtuvo una muestra de 100 datos.\n",
  "Con una eficiencia estimada de:", round(prueba$efi_global, 4), "\n",
  "Prueba KS: p-value = ", resultado_ks$p.value, "\n")

```

```

## De la función Beta creada a partir del método de aceptación
## y rechazo se obtuvo una muestra de 100 datos.
## Con una eficiencia estimada de: 0.9346
## Prueba KS: p-value = 0.9990561

```

```

# Evaluación para tamaños n = 100, 1000, 10000
tamaños <- c(100, 1000, 10000)

# Evaluamos la eficiencia y la prueba KS para cada tamaño de muestra
resultados <- lapply(tamaños, function(n_i) {
  res <- func_beta(alpha = 2.2, beta = 4.7, n = n_i)

  # Calculamos la prueba KS y extraemos solo el p-value
  ks_res <- ks.test(x = res$V, y = "pbeta", shape1 = res$alpha, shape2 = res$beta)

  data.frame(
    n = n_i,
    Propuestas = round(res$propuestas),
    Rechazadas = round(res$rechazos),
    Eficiencia_Obs = paste0(round(res$efi_global * 100, 2), "%"),
    Pvalue = round(ks_res$p.value, 4)
  )
})

# Despliegue de la tabla de resultados
tabla_eficiencia <- do.call(rbind, resultados)
knitr::kable(tabla_eficiencia, col.names = c("n", "Muestras Propuestas", "Muestras Rechazadas", "Eficiencia Observada", "P-value"))

```

n	Muestras Propuestas	Muestras Rechazadas	Eficiencia (%)	P-value KS
100	210	10	95.24%	0.7983
1000	2146	146	93.2%	0.2853
10000	21372	1372	93.58%	0.9312

Con la semilla fija, se observa como cada muestra pasa la prueba KS, por lo que es plausible considerar que la muestra proviene de una densidad $Beta(2.2, 4.7)$; la eficiencia estimada se observa que está por encima del 90%.

2. En este ejercicio se estudiará una variante para simular de una distribución normal estándar usando el método polar de Marsaglia, cuyo algoritmo es:

1. Simular $U, V \sim U(-1, 1)$.
2. Si $0 < S = U^2 + V^2 < 1$ hacer:

$$X = U \sqrt{-\frac{2 \log(S)}{S}}$$

$$Y = V \sqrt{-\frac{2 \log(S)}{S}}.$$

3. Repetir en caso contrario De esta forma al aceptar se tiene que X y Y son variables aleatorias independientes con distribución normal estándar.

(a) Implementa el algoritmo de Marsaglia y genera una muestra de tamaño $n = 10, 100, 1000, 10000$ de una distribución normal estándar. Muestra que la probabilidad teórica de aceptación del algoritmo es $\pi/4$ y compara este valor con la proporción de propuestas aceptadas obtenida en tus simulaciones.

```
func_normal <- function(n) {
  # Creamos el conjunto para X,Y
  X_totales <- c()
  Y_totales <- c()
  total_intentos <- 0

  while (length(X_totales) < n) {
    # Generamos de acuerdo a los pares que nos faltan completar
    faltantes <- ceiling((n - length(X_totales)) / (pi / 4)) # Estimación para reducir iteraciones

    U <- runif(faltantes, -1, 1)
    V <- runif(faltantes, -1, 1)

    total_intentos <- total_intentos + faltantes

    # Condición S
    S <- U^2 + V^2

    # Máscara lógica booleana
    acepta <- (S > 0) & (S < 1)

    # Filtramos U, V y S de forma independiente con la misma máscara
```

```

U_acep <- U[acepta]
V_acep <- V[acepta]
S_acep <- S[acepta]

# 5. Transformación del método polar
factor <- sqrt(-2 * log(S_acep) / S_acep)

X_nuevos <- U_acep * factor
Y_nuevos <- V_acep * factor

# 6. Acumulamos
X_totales <- c(X_totales, X_nuevos)
Y_totales <- c(Y_totales, Y_nuevos)
}

# Recortamos exactamente al tamaño n solicitado
X_final <- X_totales[1:n]
Y_final <- Y_totales[1:n]

# La muestra total de Normales Estándar independientes es de tamaño 2*n

return(list(
  X = X_final,
  Y = Y_final,
  propuestas = total_intentos * 2, # Total de U y V generados
  prop_aceptada = (length(X_totales) * 2) / (total_intentos * 2)
))
}

tamaños <- c(5, 50, 500, 5000)

# Evaluamos la eficiencia y la prueba KS para cada tamaño de muestra
resultados <- lapply(tamaños, function(n_i) {
  res <- func_normal(n = n_i)

  # Calculamos las pruebas KS
  ks_resX <- ks.test(x = res$X, y = "pnorm", mean = 0, sd = 1)
  ks_resY <- ks.test(x = res$Y, y = "pnorm", mean = 0, sd = 1)

  # Calculamos el total de rechazadas
  # Dado que cada propuesta genera un par (U, V), las propuestas aceptadas son 2*n_i
  rechazadas <- res$propuestas - (2 * n_i)

  data.frame(
    n = n_i,
    Propuestas = round(res$propuestas),
    Rechazadas = round(rechazadas),
    Prop_Aceptada = paste0(round(res$prop_aceptada * 100, 2), "%"),
    Pvalue_X = round(ks_resX$p.value, 4),
    Pvalue_Y = round(ks_resY$p.value, 4)
  )
})

```

```
# Despliegue de la tabla de resultados
tabla_result <- do.call(rbind, resultados)
knitr::kable(
  tabla_result,
  col.names = c("n", "Muestras Propuestas", "Muestras Rechazadas", "Prop. Aceptación", "p-value (X)", "p-value (Y)", " ")
)
```

n	Muestras Propuestas	Muestras Rechazadas	Prop. Aceptación	p-value (X)	p-value (Y)
5	14	4	85.71%	0.0845	0.7783
50	128	28	81.25%	0.3373	0.0705
500	1296	296	77.31%	0.2203	0.5224
5000	12734	2734	79.13%	0.2224	0.7828

En la anterior tabla se muestra el número de intentos, las veces que se rechazaron, el p_value para las dos variables normales, se hizo un arreglo de la muestra requerida entre 2, debido a que el algoritmo nos genera dos muestras del mismo tamaño, se observa en la columna de proporción aceptada (Que es lo mismo que la probabilidad clásica), que mediante va aumentando n , la proporción se aproxima a $\frac{\pi}{4} \approx 0.7854$.

(b) Compara tu algoritmo contra el método de Box-Muller discutido en clase y el algoritmo por default de R para los mismos tamaños de muestra $n = 10, 100, 1000, 10000$. ¿Cuál es más eficiente para cada caso? Recordemos que el algoritmo de Box-Muller consiste en un cambio de variable de la manera: A partir de $U_1, U_2 \sim U(0, 1)$ independientes: Radio (R):

$$R = \sqrt{-2 \ln(U_1)}$$

Ángulo (Θ):

$$\Theta = 2\pi U_2$$

El cambio de variable para obtener (X, Y) es:

$$X = R \cos(\Theta) = \sqrt{-2 \ln(U_1)} \cos(2\pi U_2)$$

$$Y = R \sin(\Theta) = \sqrt{-2 \ln(U_1)} \sin(2\pi U_2)$$

Implementando el algoritmo:

```
# Definimos la funcion
box_muller <- function(n) {
  # Aplicamos funcion techo
  k <- ceiling(n / 2)

  # Transformación de Box-Muller
  R <- sqrt(-2 * log(runif(k)))
  Theta <- 2 * pi * runif(k)

  X <- R * cos(Theta)
  Y <- R * sin(Theta)

  # Concatenamos y recortamos exactamente a 'n' elementos
  muestra <- c(X, Y)[1:n]

  return(muestra)
}
```

```

tamaños <- c(5, 50, 500, 5000)

# Evaluamos la eficiencia y la prueba KS para cada tamaño de muestra
resultados <- lapply(tamaños, function(n_i) {
  res <- box_muller(n = n_i)

  # Separamos el vector concatenado en X e Y para hacer la prueba KS a cada uno
  k <- ceiling(n_i / 2)
  X_val <- res[1:k]
  Y_val <- res[(k + 1):n_i]

  # Calculamos las pruebas KS
  ks_resX <- ks.test(x = X_val, y = "pnorm", mean = 0, sd = 1)

  # Si n es muy pequeño
  # Evaluamos Y solo si hay suficientes datos
  if (length(Y_val) > 0) {
    ks_resY <- ks.test(x = Y_val, y = "pnorm", mean = 0, sd = 1)
    p_val_y <- round(ks_resY$p.value, 4)
  } else {
    p_val_y <- NA
  }

  # Lo presentamos como Data Frame
  data.frame(
    n = n_i,
    Pvalue_X = round(ks_resX$p.value, 4),
    Pvalue_Y = p_val_y
  )
})

# Despliegue de la tabla de resultados
tabla_result <- do.call(rbind, resultados)
knitr::kable(
  tabla_result,
  col.names = c("n", "p-value (X)", "p-value (Y)")
)

```

n	p-value (X)	p-value (Y)
5	0.7170	0.9829
50	0.0336	0.4673
500	0.5957	0.3324
5000	0.7378	0.8194

Se observa que la prueba de bondad de ajuste para una $N(0,1)$ lo pasa; debido a que el algoritmo de Marsaglia tiene una tasa de aceptación de $\frac{\pi}{4}$ y la de Boc-Muller de 1; se hará la comparación de eficiencia a nivel computacional, con el uso de la librería “microbenchmark”, se hará la combinación de generar cada muestra del mismo tamaño:

```

tamaños <- c(5, 50, 500, 5000)

```

```

# Iteramos microbenchmark para cada tamaño n
benchmarks_list <- lapply(tamaños, function(n_i) {
  bm <- microbenchmark(
    Box_Muller = box_muller(n = n_i),
    Marsaglia = func_normal(n = n_i),
    times = 50 # Número de repeticiones por cada prueba
  )

  # Extraemos el resumen estadístico de la ejecución
  res <- summary(bm)
  res$n <- n_i
  return(res)
})

# Consolidador de resultados
res_benchmark <- do.call(rbind, benchmarks_list)

# Formato visual en tabla (los tiempos se muestran en microsegundos/milisegundos según escala)
knitr::kable(
  res_benchmark[, c("n", "expr", "min", "mean", "median", "max")],
  col.names = c("n", "Método", "Mínimo", "Media", "Mediana", "Máximo"),
  digits = 3
)

```

n	Método	Mínimo	Media	Mediana	Máximo
5	Box_Muller	2.0	2.308	2.20	6.6
5	Marsaglia	3.5	5.298	3.90	48.0
50	Box_Muller	4.5	6.248	5.20	25.7
50	Marsaglia	7.7	13.506	11.95	32.7
500	Box_Muller	27.5	34.416	29.40	144.7
500	Marsaglia	36.3	47.282	44.90	96.2
5000	Box_Muller	259.6	315.300	310.85	450.6
5000	Marsaglia	327.8	550.624	549.85	1773.1

En general, a pesar de que el método de Box-Muller hace uso de más funciones trascendentes que la de Marsaglia, al hacer mi función vectorizada me reduce el tiempo de ejecución a que si iterada en los casos que pasa en Marsaglia.

#3. Considera una urna con bolas de K diferentes colores donde, al tiempo 0, hay α_i bolas del i -ésimo color. Al tiempo n , se saca una bola y se regresa a la urna junto con una nueva bola de ese color, repitiendo el proceso N veces. Si $N \rightarrow \infty$, se puede demostrar que el vector de proporciones de bolas en la urna converge casi seguramente a un vector aleatorio con distribución Dirichlet. (a) Considerando $K = 3$, $\alpha = (2, 5, 1)$ y un valor de N “suficientemente grande”, implementa este algoritmo y obtén una muestra de tamaño $n = 10, 50, 100, 1000$. Para diferentes valores de N (por ejemplo: 100, 1000, 10000, 100000), compara la media, el segundo momento y la covarianza muestral contra los momentos teóricos de la distribución. Debido a que ya tenemos nuestros valores iniciales en la urna para los 3 colores, basta con hacer una función que implemente el algoritmo mencionado en las instrucciones.

```

dirichlet1 <- function(n_extracciones, alpha) {
  k <- length(alpha)
  # Me crea mi vecotr inicial asociando número(1,2,3) a los colores y
  # repite el número alpha[i] veces entradas seria c(1,1,2,2,2,2,2,3)

```

```

urna <- rep(1:k, times = alpha)
# Simulamos la extracción
for (i in 1:k){
  canica_sacada <- sample(urna, size = 1)
  # Agregamos una canica del mismo tipo
  urna <- c(urna, canica_sacada)
}
# Calculo de las proporciones (Pongo table para que me agrupe los datos)
proporciones <- table(factor(urna, levels = 1:k)) / length(urna)
return(as.vector(proporciones))
}
dirichlet1(n_extracciones = 10000, alpha = c(2,5,1))

```

```
## [1] 0.1818182 0.6363636 0.1818182
```

Ya tenemos una función que nos da 1 sola muestra de una dirichlet con cierto α , usaremos una función llamada replicate para obtener una muestra de tamaño n.

```

muestra_dirichlet1 <- function(n, alpha, N) {
  # Usamos replicate para generar la muestra
  muestra_list <- replicate(n, dirichlet1(N, alpha))

  # Acomodamos los valores en el data frame
  muestra_matriz <- t(muestra_list)
  colnames(muestra_matriz) <- paste0("p_", 1:length(alpha))
  muestra_df <- as.data.frame(muestra_matriz)

  # Momentos muestrales
  media_m <- colMeans(muestra_df)
  segundo_momento_m <- colMeans(muestra_df^2)
  covarianza_m <- cov(muestra_df)

  # Momentos poblacionales de una Dirichlet
  alpha_0 <- sum(alpha)

  media_t <- alpha / alpha_0
  segundo_momento_t <- (alpha * (alpha + 1)) / (alpha_0 * (alpha_0 + 1))

  covarianza_t <- - (alpha %*% t(alpha)) / (alpha_0^2 * (alpha_0 + 1))

  diag(covarianza_t) <- (alpha * (alpha_0 - alpha)) / (alpha_0^2 * (alpha_0 + 1))

  colnames(covarianza_t) <- paste0("p_", 1:length(alpha))
  rownames(covarianza_t) <- paste0("p_", 1:length(alpha))

  return(list(
    muestra = muestra_df,
    media_muestral = media_m,
    media_teorica = media_t,
    segundo_momento_muestral = segundo_momento_m,
    segundo_momento_teorico = segundo_momento_t,
  ))
}

```

```

    covarianza_muestral = covarianza_m,
    covarianza_teorica = covarianza_t
  ))
}
muestra_dirichlet1(n=10, alpha = c(2,5,1), N=10000)

```

```

## $muestra
##      p_1      p_2      p_3
## 1 0.1818182 0.7272727 0.09090909
## 2 0.1818182 0.6363636 0.18181818
## 3 0.2727273 0.6363636 0.09090909
## 4 0.2727273 0.4545455 0.27272727
## 5 0.1818182 0.7272727 0.09090909
## 6 0.2727273 0.6363636 0.09090909
## 7 0.2727273 0.6363636 0.09090909
## 8 0.2727273 0.6363636 0.09090909
## 9 0.3636364 0.5454545 0.09090909
## 10 0.1818182 0.7272727 0.09090909
##
## $media_muestral
##      p_1      p_2      p_3
## 0.2454545 0.6363636 0.1181818
##
## $media_teorica
## [1] 0.250 0.625 0.125
##
## $segundo_momento_muestral
##      p_1      p_2      p_3
## 0.06363636 0.41157025 0.01735537
##
## $segundo_momento_teorico
## [1] 0.08333333 0.41666667 0.02777778
##
## $covarianza_muestral
##      p_1      p_2      p_3
## p_1 3.764922e-03 -0.003673095 -9.182736e-05
## p_2 -3.673095e-03 0.007346189 -3.673095e-03
## p_3 -9.182736e-05 -0.003673095 3.764922e-03
##
## $covarianza_teorica
##      p_1      p_2      p_3
## p_1 0.020833333 -0.017361111 -0.003472222
## p_2 -0.017361111 0.026041667 -0.008680556
## p_3 -0.003472222 -0.008680556 0.012152778

```

Se puede comparar los estimadores y los valores reales, se calculará para valores requeridos de n :

```

tamanos <- c(10, 50, 100, 1000)
tamano_N <- c(100, 1000, 10000, 100000)
alpha <- c(2, 5, 1)

resultados_list <- mapply(function(n_i, N_i) {
  res <- muestra_dirichlet1(n = n_i, alpha = alpha, N = N_i)

```

```

data.frame(
  n = n_i,
  N = N_i,
  media_muestral = paste(round(res$media_muestral, 3), collapse = ", "),
  media_teorica = paste(round(res$media_teorica, 3), collapse = ", "),
  seg_momento_m = paste(round(res$segundo_momento_muestral, 3), collapse = ", "),
  seg_momento_t = paste(round(res$segundo_momento_teorico, 3), collapse = ", "),
  covarianza_m = paste(round(as.vector(res$covarianza_muestral), 3), collapse = ", "),
  covarianza_t = paste(round(as.vector(res$covarianza_teorica), 3), collapse = ", ")
)
}, n_i = tamanos, N_i = tamaño_N, SIMPLIFY = FALSE)

# 1. Unimos los resultados
tabla_result <- do.call(rbind, resultados_list)

# Para evitar el fallo de Pandoc al correr el chunk individualmente:
print(as.data.frame(tabla_result))

```

```

##      n      N      media_muestral      media_teorica      seg_momento_m
## 1   10 1e+02 0.218, 0.636, 0.145 0.25, 0.625, 0.125 0.05, 0.413, 0.026
## 2   50 1e+03 0.253, 0.62, 0.127 0.25, 0.625, 0.125 0.069, 0.392, 0.02
## 3  100 1e+04 0.256, 0.62, 0.124 0.25, 0.625, 0.125 0.072, 0.392, 0.019
## 4 1000 1e+05 0.248, 0.629, 0.123 0.25, 0.625, 0.125 0.067, 0.403, 0.018
##      seg_momento_t
## 1 0.083, 0.417, 0.028
## 2 0.083, 0.417, 0.028
## 3 0.083, 0.417, 0.028
## 4 0.083, 0.417, 0.028
##
##                                     covarianza_m
## 1  0.002, -0.003, 0.001, -0.003, 0.009, -0.006, 0.001, -0.006, 0.006
## 2 0.005, -0.005, -0.001, -0.005, 0.008, -0.003, -0.001, -0.003, 0.004
## 3 0.007, -0.006, -0.001, -0.006, 0.008, -0.002, -0.001, -0.002, 0.004
## 4 0.006, -0.005, -0.001, -0.005, 0.007, -0.002, -0.001, -0.002, 0.003
##
##                                     covarianza_t
## 1 0.021, -0.017, -0.003, -0.017, 0.026, -0.009, -0.003, -0.009, 0.012
## 2 0.021, -0.017, -0.003, -0.017, 0.026, -0.009, -0.003, -0.009, 0.012
## 3 0.021, -0.017, -0.003, -0.017, 0.026, -0.009, -0.003, -0.009, 0.012
## 4 0.021, -0.017, -0.003, -0.017, 0.026, -0.009, -0.003, -0.009, 0.012

```

Se tiene los resultados para todos los casos. (b) Realiza el mismo proceso pero ahora a partir de la generación de variables Dirichlet utilizando la construcción a partir de las variables Gamma vista en clase. Es igual de sencillo, ocuparemos la función “rgamma” que es genérica de R para obtener nuestra muestra de una Dirichlet.

```

dirichlet2 <- function(alpha) {
  y <- rgamma(n = length(alpha), shape = alpha, rate = 1)
  return(y/sum(y))
}
dirichlet2(alpha = c(2,5,1))

```

```
## [1] 0.24585702 0.68671240 0.06743058
```

El código anterior nos dió una muestra de tamaño uno de un proceso Dirichlet a partir del uso de funciones de densidad Gammas, se creará la función para generar una muestra de tamaño n de esa función.

```
muestra_dirichlet2 <- function(n, alpha) {
  # Generar la muestra
  muestra_list <- replicate(n, dirichlet2(alpha))

  # Acomodar como Data Frame
  muestra_matriz <- t(muestra_list)
  colnames(muestra_matriz) <- paste0("p_", 1:length(alpha))
  muestra_df <- as.data.frame(muestra_matriz)

  # Momentos muestrales
  media_m <- colMeans(muestra_df)
  segundo_momento_m <- colMeans(muestra_df^2)
  covarianza_m <- cov(muestra_df)

  # Momentos teóricos
  alpha_0 <- sum(alpha)
  media_t <- alpha / alpha_0
  segundo_momento_t <- (alpha * (alpha + 1)) / (alpha_0 * (alpha_0 + 1))

  covarianza_t <- - (alpha %*% t(alpha)) / (alpha_0^2 * (alpha_0 + 1))
  diag(covarianza_t) <- (alpha * (alpha_0 - alpha)) / (alpha_0^2 * (alpha_0 + 1))
  colnames(covarianza_t) <- paste0("p_", 1:length(alpha))
  rownames(covarianza_t) <- paste0("p_", 1:length(alpha))

  return(list(
    muestra = muestra_df,
    media_muestral = media_m,
    media_teorica = media_t,
    segundo_momento_muestral = segundo_momento_m,
    segundo_momento_teorico = segundo_momento_t,
    covarianza_muestral = covarianza_m,
    covarianza_teorica = covarianza_t
  ))
}

# Ejecución de prueba
res2 <- muestra_dirichlet2(n = 10, alpha = c(2, 5, 1))
print(res2)
```

```
## $muestra
##      p_1      p_2      p_3
## 1 0.09855709 0.8682004 0.033242532
## 2 0.14514224 0.8230157 0.031842066
## 3 0.05001950 0.7055690 0.244411515
## 4 0.01466150 0.9406248 0.044713671
## 5 0.15772285 0.8171939 0.025083267
## 6 0.23057232 0.6590768 0.110350918
## 7 0.15962719 0.8333533 0.007019529
## 8 0.30905102 0.5054212 0.185527810
## 9 0.41864342 0.4188581 0.162498481
## 10 0.24763076 0.7485827 0.003786522
```

```
##
## $media_muestral
##      p_1      p_2      p_3
## 0.18316279 0.73198958 0.08484763
##
## $media_teorica
## [1] 0.250 0.625 0.125
##
## $segundo_momento_muestral
##      p_1      p_2      p_3
## 0.04691135 0.56016638 0.01375517
##
## $segundo_momento_teorico
## [1] 0.08333333 0.41666667 0.02777778
##
## $covarianza_muestral
##      p_1      p_2      p_3
## p_1 0.014847486 -0.01731351 0.002466028
## p_2 -0.017313515 0.02706404 -0.009750530
## p_3 0.002466028 -0.00975053 0.007284501
##
## $covarianza_teorica
##      p_1      p_2      p_3
## p_1 0.020833333 -0.017361111 -0.003472222
## p_2 -0.017361111 0.026041667 -0.008680556
## p_3 -0.003472222 -0.008680556 0.012152778
```

Con la función anterior podemos obtener una muestra Dirichlet de tamaño deseado. (c) *Compara los resultados obtenidos mediante ambos algoritmos. ¿Cómo afecta N a la aproximación obtenida mediante la urna de Pólya? ¿Qué diferencias observas entre ambos métodos en términos de precisión y costo computacional?*

```
tamanos <- c(10, 50, 100, 1000)
tamano_rep <- c(100, 1000, 10000, 100000)
alpha_vec <- c(2, 5, 1)

# Iteramos sobre n_i y N_i en paralelo
benchmarks_list <- mapply(function(n_i, N_i) {
  bm <- microbenchmark(
    Urna_Polya = muestra_dirichlet1(n = n_i, alpha = alpha_vec, N = N_i),
    Metodo_Gamma = muestra_dirichlet2(n = n_i, alpha = alpha_vec),
    times = 20 # Reducimos repeticiones para agilizar ejecuciones con N grande
  )

  # Extraemos el resumen
  res <- summary(bm)
  res$n <- n_i
  res$N <- N_i
  return(res)
}, n_i = tamanos, N_i = tamano_rep, SIMPLIFY = FALSE)

# Consolidador de resultados
res_benchmark <- do.call(rbind, benchmarks_list)

# Despliegue en consola/tabla
```

```
print(as.data.frame(res_benchmark[, c("n", "N", "expr", "min", "mean", "median", "max"))))
```

##	n	N	expr	min	mean	median	max
## 1	10	1e+02	Urna_Polya	621.8000	672.370000	652.90000	918.2000
## 2	10	1e+02	Metodo_Gamma	286.0000	324.530000	321.70000	376.0000
## 3	50	1e+03	Urna_Polya	1779.7000	2026.830000	1942.00000	3680.1000
## 4	50	1e+03	Metodo_Gamma	363.7000	438.540000	424.90000	599.0000
## 5	100	1e+04	Urna_Polya	3175.5000	3558.800000	3432.00000	5093.6000
## 6	100	1e+04	Metodo_Gamma	480.2000	547.905000	529.55000	723.0000
## 7	1000	1e+05	Urna_Polya	30.6262	34.470475	34.17335	42.1633
## 8	1000	1e+05	Metodo_Gamma	2.3157	2.543785	2.45540	4.1052

Se observa que la eficiencia y el que tiene menos costo computacional es el algoritmo con uso de gammas y no el de la Urna de Polya, al tener que el proceso Dirichlet es consecuencia de una distribución asintótica de la Urna de Polya, se necesitarían un tamaño suficientemente grande de N para tener buena aproximación, se observa como el tiempo de la Urna de Polya tiene una marcada disminución cuando $N = 100000$, pero esto se debe a un cambio de escala, lo que indica que aún así el tiempo sigue aumentando, lo mismo pasa con el método de las Gammas pero el tiempo de respuesta es inferior al de la Urna basandonos en el data frame generado, esto se debe a que el proceso Dirichlet apartir de Gammas está completamente vectorizado, lo que reduce mucho el costo computacional.